Week 2 at a glance

We will be learning and practicing to:

- Model systems with tools from discrete mathematics and reason about implications of modelling choices. Explore applications in CS through multiple perspectives, including software, hardware, and theory.
  - Selecting and representing appropriate data types and using notation conventions to clearly communicate choices
  - Determining the properties of positional number representations, including overflow and bit operations
- Translate between different representations to illustrate a concept.
  - Translating between symbolic and English versions of statements using precise mathematical language
  - Tracing algorithms specified in pseudocode
  - Representing numbers using positional representations, including decimal, binary, hexadecimal, fixed-width representations, and 2s complement
- Use precise notation to encode meaning and present arguments concisely and clearly
  - Precisely describing a set using appropriate notation e.g. roster method, set builder notation, and recursive definitions
  - Defining functions using multiple representations
- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
  - Using the definitions of the div and mod operators on integers

TODO:

#FinAid Assignment on Canvas (complete as soon as possible)

Review quiz based on Week 0 and 1 class material (due Monday)

Homework assignment 1 (due Tuesday)

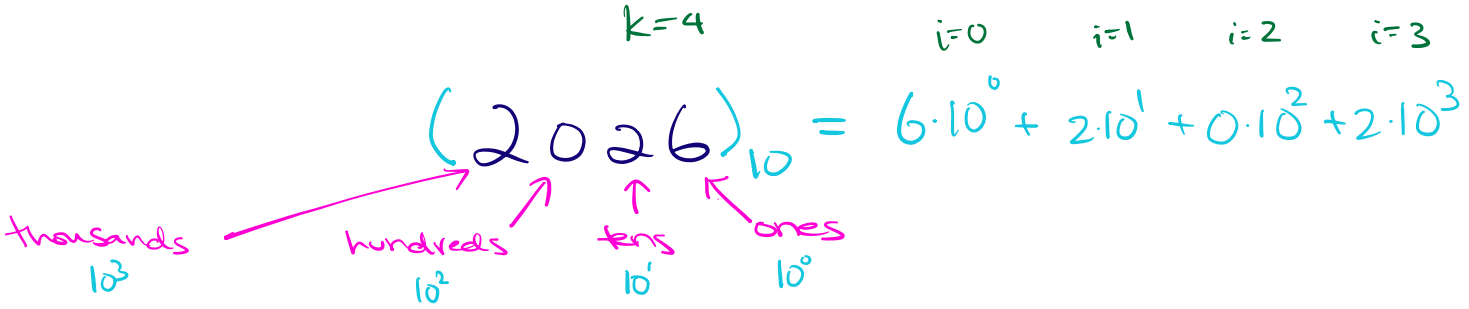
Review quiz based on Week 2 class material (due Monday next week)

Discussion notes from Monday on Week 2 Learning Material section of website.

CC BY-NC-SA 2.0

Week 2 Monday: Representing numbers

Modeling uses data-types that are encoded in a computer. The details of the encoding impact the efficiency of algorithms we use to understand the systems we are modeling and the impacts of these algorithms on the people using the systems. Case study: how to encode numbers?



Definition For b an integer greater than 1 and n a positive integer, the **base b expansion of n** is

$$(a_{k-1} \cdots a_1 a_0)_b$$

where k is a positive integer, $a_0, a_1, \ldots, a_{k-1}$ are (symbols for) nonnegative integers less than b, $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i b^i$$

$$= a_{k-1} b^{k-1} + \cdots + a_1 b^1 + a_0 b^0$$

Notice: *The base b expansion of a positive integer n is a string over the alphabet $\{x \in \mathbb{N} \mid x < b\}$ whose leftmost character is nonzero.*

Base b	Collection of possible coefficients in base b expansion of a positive integer
Binary ($b = 2$)	$\{0, 1\}$
Ternary ($b = 3$)	$\{0, 1, 2\}$
Octal ($b = 8$)	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Decimal ($b = 10$)	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Hexadecimal ($b = 16$)	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ letter coefficient symbols represent numerical values $(A)_{16} = (10)_{10}$ $(B)_{16} = (11)_{10}$ $(C)_{16} = (12)_{10}$ $(D)_{16} = (13)_{10}$ $(E)_{16} = (14)_{10}$ $(F)_{16} = (15)_{10}$

CC BY-NC-SA 2.0

Examples:

1401 = ( )$_2$

~~$(1401)_2$~~ Definition of Binary expansions requires only 0 and 1 coefficient (and leftmost coefficient must be 1).

$(1401)_{10} = 1\cdot 10^3 + 4\cdot 10^2 + 0\cdot 10^1 + 1\cdot 10^0$

ones

thousands

10^3 $16^3 = 4096$ $16^2 = 256$ $16^1 = 16$ $16^0 = 1$

$(1401)_{16} = 1\cdot 16^3 + 4\cdot 16^2 + 0\cdot 16^1 + 1\cdot 16^0$

ones

16^3

$= 1\cdot 4096 + 4\cdot 256 + 0 + 1$

$= (5121)_{10}$

Another example

"A" = "10"

$(A1)_{16}$

16^1 16^0

$= 10\cdot 16^1 + 1\cdot 16^0$

$= 160 + 1$

$= (161)_{10}$

Definition For b an integer greater than 1, w a positive integer, and n a nonnegative integer $n < b^w$, the **base b fixed-width w expansion of** n is

fixed # columns. b^{w-1} b^1 b^0 ≥ 0

$(a_{w-1}\cdots a_1 a_0)_{b,w}$

where $a_0, a_1, \ldots, a_{w-1}$ are nonnegative integers less than b and

$$n = \sum_{i=0}^{w-1} a_i b^i$$

Base 2 expansion

Decimal $b = 10$	Binary $b = 2$	Binary fixed-width 10 $b = 2$, $w = 10$	Binary fixed-width 7 $b = 2$, $w = 7$	Binary fixed-width 4 $b = 2$, $w = 4$
$(20)_{10}$	$(10100)_2$	$(0000010100)_{2,10}$	$(0010100)_{2,7}$	

2^4 2^3 2^2 2^1 2^0

$20 = 16 + 4 = 2^4 + 2^2$

MAX: $(1111)_{2,4}$

$= 8 + 4 + 2 + 1$

$= 15$

CC BY-NC-SA 2.0 Version October 3, 2026 (3)

Definition For b an integer greater than 1, w a positive integer, w' a positive integer, and x a real number the **base b fixed-width expansion of x with integer part width w and fractional part width w'** is $(a_{w-1} \cdots a_1 a_0 . c_1 \cdots c_{w'})_{b,w,w'}$ where $a_0, a_1, \ldots, a_{w-1}, c_1, \ldots, c_{w'}$ are nonnegative integers less than b and

$$x \geq \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} \quad \text{and} \quad x < \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} + b^{-w'}$$

3.75 in fixed-width binary, integer part width 2, fractional part width 8 $3.75 \geq 3.75$ $3.75 < 3.7539625$	radix point $(1\,1.1\,1\,0\,0\,0\,0\,0)_{2,2,8}$ $2^1\ 2^0\ 2^{-1}\ 2^{-2}$ $b^{-w'} = 2^{-8} = 0.00390625$
0.1 in fixed-width binary, integer part width 2, fractional part width 8 $0.1 \geq \frac{1}{16} + \frac{1}{32} + \frac{1}{256} = 0.09765625$ $0.1 < \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \frac{1}{256} = 0.1015625$	$(0\,0.0\,0\,0\,1\,1\,0\,0\,1)_{2,2,8}$ $2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}\ 2^{-5}\ 2^{-6}\ 2^{-7}\ 2^{-8}$ $0.1 < 0.5$ $0.1 < 0.25$ $0.1 < 0.125$ 0.0625 0.03125 $0.1000 - 0.0625 = 0.0375$ $0.0375 - 0.03125 = 0.00625$

$\frac{1}{2} = 0.5$ $\frac{1}{4} = 0.25$ $\frac{1}{8} = 0.125$ $\frac{1}{16} = 0.0625$ $\frac{1}{32} = 0.03125$ $\frac{1}{64} = 0.015625$ $\frac{1}{128} = 0.0078125$ $\frac{1}{256} = 0.00390625$

```
[welcome $jshell
|  Welcome to JShell -- Version 10.0.1
|  For an introduction type: /help intro

[jshell> 0.1
$1 ==>

[jshell> 0.2
$2 ==>

[jshell> 0.1 + 0.2
$3 ==>

[jshell> Math.sqrt(2)
$4 ==>

[jshell> Math.sqrt(2)*Math.sqrt(2)
$5 ==>

jshell>
```

Note: Java uses floating point, not fixed width representation, but similar rounding errors appear in both.

Extra Practice:

Representing 0: Which of the representations so far allow us to represent 0?

CC BY-NC-SA 2.0

Base b expansion of n

- represent positive integer
- unique representation (for specific choice b)

Base b fixed-width w expansion of n

- represent nonnegative integers $< b^w$
- unique representation (for specific choices of b, w)

Base b fixed-width w expansion of x with integer part width w and fractional part width w'

- representing real numbers in interval  depend on b, w, w'
- each number in this interval has an expansion
- one expansion represents multiple numbers in this interval

$(1.00)_{2,1,2}$

Week 2 Wednesday: Algorithms for numbers

Find and fix any and all mistakes with the following:

(a) $(1)_2 = (1)_8$ ✓

(b) $(142)_{10} \neq (142)_{16}$ $(142)_{16} = 1\cdot16^2 + 4\cdot16^1 + 2\cdot16^0 = 256 + 64 + 2 = 322$

(c) $(20)_{10} = (10100)_2$ ✓ $(10100)_2 = 1\cdot2^4 + 0\cdot2^3 + 1\cdot2^2 + 0\cdot2^1 + 0\cdot2^0 = 16 + 4 = 20$ (16 8 4 2 1)

(d) $(35)_8 = (1D)_{16}$ ✓ $(35)_8 = 3\cdot8^1 + 5\cdot8^0 = 24 + 5 = 29$; $(1D)_{16} = 1\cdot16^1 + 13\cdot16^0 = 16 + 13 = 29$

extra practice

New! An algorithm is a finite sequence of precise instructions for solving a problem.

Algorithms can be expressed in English or in more formalized descriptions like pseudocode or fully executable programs.

Sometimes, we can define algorithms whose output matches the rule for a function we already care about. Consider the (integer) logarithm function

$$logb : \{b \in \mathbb{Z} \mid b > 1\} \times \mathbb{Z}^+ \to \mathbb{N}$$

Domain Codomain

defined by

$$logb((b, n)) = \text{greatest integer } y \text{ so that } b^y \text{ is less than or equal to } n$$

Calculating integer part of base b logarithm

```
1  procedure logb(b,n: positive integers with b>1)
2  i := 0                                  defining variable
3  while n > b-1                           loop
4    i := i+1
5    n := n div b
6  return i {i holds the integer part of the base b logarithm of n}   output
```

Trace this algorithm with inputs $b = 3$ and $n = 17$

	b	n	i	$n > b - 1$?
Initial value	3	17	0	17 > 2 ? T
After 1 iteration		5	1	5 > 2 ? T
After 2 iterations		1	2	1 > 2 ? F
~~After 3 iterations~~				

17 div 3 = 5 b/c 17 = 5 · 3 + 2 (q = 5, r = 2)

5 div 3 = 1 b/c 5 = 1 · 3 + 2 (q = 1, r = 2)

Return 2

logb((3,17))

= greatest integer so that b^y is less than or equal to 17.

= 2 b/c $3^2 \leq 17$ but $3^3 > 17$.

Compare: does the output match the rule for the (integer) logarithm function?

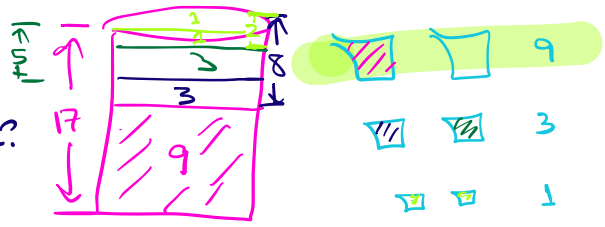
CC BY-NC-SA 2.0

Two algorithms for constructing base b expansion from decimal representation

Most significant first: Start with left-most coefficient of expansion (highest value)

Informally: Build up to the value we need to represent in "greedy" approach, using units determined by base.

Base b: b^{k-1} ... b^1 b^0



Example: What's base 3 expansion of 17?

$(\underset{9}{1}\ \underset{3}{2}\ \underset{1}{2})_3$

$17 \stackrel{?}{=} (1\,22)_3 = 1\cdot 3^2 + 2\cdot 3^1 + 2\cdot 3^0 = 9+6+2 = 17$ ✓

Calculating base b expansion, from left

```
1  procedure basebmost(n, b: positive integers with b > 1)
2  v := n
3  k := 1+ output of logb algorithm with inputs b and n
4  for i := 1 to k
5     a_{k-i} := 0
6     while v ≥ b^{k-i}
7        a_{k-i} := a_{k-i} + 1
8        v := v − b^{k-i}
9  return (a_{k-1}, ..., a_0){(a_{k-1} ... a_0)_b is the base b expansion of n}
```

value remaining; # of coefficients in expansion

	n	b	v	k	i	a_{k-i}	$v \geq b^{k-i}$?
initial value	17	3	17	3	1		
i=1 in loop			~~17~~ 8		1	a_2 = ~~0~~ 1	$17 \geq 3^2$? T $8 \geq 3^2$? F
i=2 in loop			~~17~~ ~~8~~ 2		2	a_1 = ~~0~~ ~~1~~ 2	$8 \geq 3^1$? T $5 \geq 3^1$? T $2 \geq 3^1$? F
i=3 in loop			~~17~~ ~~8~~ 0		3	a_0 = ~~0~~ ~~1~~ 2	$2 \geq 3^0$? T $1 \geq 3^0$? T $0 \geq 3^0$? F

Return (1,2,2)

Corresponding to $(122)_3$

MOST SIGNIFICANT COEFFICIENT → a_2 a_1 a_0

CC BY-NC-SA 2.0

Least significant first: Start with right-most coefficient of expansion (lowest value)

Idea: (when $k > 1$)

$$\begin{aligned} n &= a_{k-1}b^{k-1} + \cdots + a_1 b + a_0 \\ &= b(a_{k-1}b^{k-2} + \cdots + a_1) + a_0 \end{aligned}$$

so $a_0 = n$ **mod** b and $a_{k-1}b^{k-2} + \cdots + a_1 = n$ **div** b.

Calculating base b expansion, from right

```
1 procedure basebleast(n,b: positive integers with b>1)
2 k := 0
3 q := n
4 while q ≠ 0
5    a_k := q mod b
6    k := k+1
7    q := q div b
8 return (a_{k-1},...,a_0){(a_{k-1}...a_0)_b is the base b expansion of n}
```

UPDATED

Base 3 expansion of 17 $n=17$ $b=3$

$a_0 = 17 \bmod 3 = 2$ $17 = 5 \cdot 3 + 2$ — — a_0

LEAST SIGNIFICANT COEFFICIENT

$a_1 = 5 \bmod 3 = 2$ $5 = 1 \cdot 3 + 2$

$a_2 = 1 \bmod 3 = 1$ $1 = 0 \cdot 3 + 1$

$(1\ 2\ 2)_3 = 17$

$n=17$ $b=3$

	k	q	a_k	$q \neq 0$?
Initial	0	17		T
After 1 iteration	1	5	$a_0 = 2$	T
After 2 iterations	2	1	$a_1 = 2$	T
After 3 iterations	3	0	$a_2 = 1$	F

Output (1,2,2)
Corresponding to $(122)_3$

CC BY-NC-SA 2.0

Practice: write an algorithm for converting from base b_1 expansion to base b_2 expansion:

In English...

Inputs

Given a base b_1 and an expansion $(a_{k-1} \cdots a_0)_{b_1}$ and a target base b_2 (where b_1, b_2 both positive integers greater than 1)

$b_1 \in \mathbb{Z}^{>1}$

$b_2 \in \{x \in \mathbb{Z} \mid x > 1\}$

1. Calculate $n = \sum_{i=0}^{k-1} a_i b_1^i$
2. Use baseBleast algorithm with inputs n and b_2 to calculate the base b_2 expansion of n
3. Output the result of this algorithm, this expansion.

CC BY-NC-SA 2.0 Version October 3, 2026 (8)

Week 2 Friday: Numbers to circuits

coefficients: 0, 1

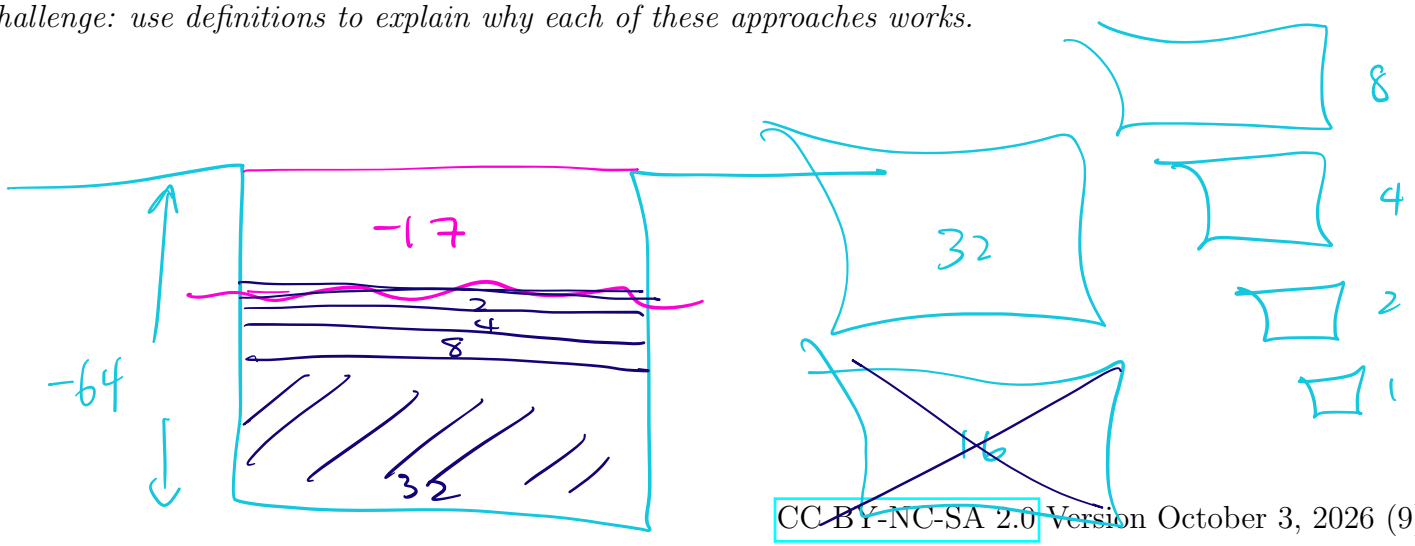
Representing negative integers in binary: Fix a positive integer width for the representation w, $w > 1$.

	To represent a positive integer n	To represent a negative integer $-n$
Sign-magnitude	$[0a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 17$, $w = 7$: $17 = [0\ 0\ 1\ 0\ 0\ 0\ 1]_{s,7}$ (sign, 32, 16, 8, 4, 2, 1)	$[1a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = -17$, $w = 7$: $-17 = [1\ 0\ 1\ 0\ 0\ 0\ 1]_{s,7}$ (sign, 32, 16, 8, 4, 2, 1)
2s complement	$[0a_{w-2} \cdots a_0]_{2c,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $n = 17$, $w = 7$: $17 = [0\ 0\ 1\ 0\ 0\ 0\ 1]_{2c,7}$ (−64, 32, 16, 8, 4, 2, 1)	$[1a_{w-2} \cdots a_0]_{2c,w}$, where $2^{w-1} - n = (a_{w-2} \cdots a_0)_{2,w-1}$ Example $-n = -17$, $w = 7$: $-17 = [1\ 1\ 0\ 1\ 1\ 1\ 1]_{2c,7}$ (−64, 32, 16, 8, 4, 2, 1)

For positive integer n, to represent $-n$ in 2s complement with width w,

- Calculate $2^{w-1} - n$, convert result to binary fixed-width $w - 1$, pad with leading 1, or
- Express $-n$ as a sum of powers of 2, where the leftmost 2^{w-1} is negative weight, or
- Convert n to binary fixed-width w, flip bits, add 1 (ignore overflow)

Challenge: use definitions to explain why each of these approaches works.



CC BY-NC-SA 2.0 Version October 3, 2026 (9)

Fixed-width addition: adding one bit at time, using the usual column-by-column and carry arithmetic, and dropping the carry from the leftmost column so the result is the same width as the summands. *Does this give the right value for the sum?*

$$\begin{array}{r} {\scriptstyle 1}\downarrow \\ [2\ 6]_{10,2} \\ +[8\ 4]_{10,2} \\ \hline [1\ 0]_{10,2} \end{array}$$

uh oh.

overflow

Base 2 fixed width 6 expansion:

$$\begin{array}{r} (1\ 1\ 0\ 1\ 0\ 0)_{2,6} \\ +(0\ 0\ 0\ 1\ 0\ 1)_{2,6} \\ \hline 1\ 1\ 1\ 0\ 0\ 1 \end{array}$$

52 + 5 = 57

expected result? ✓

Sign magnitude width 6 expansion:

$$\begin{array}{r} [1\ 1\ 0\ 1\ 0\ 0]_{s,6} \\ +[0\ 0\ 0\ 1\ 0\ 1]_{s,6} \\ \hline 1\ 1\ 1\ 0\ 0\ 1 \end{array}$$

(16 8)

−20 + 5 = −25

expected result? ✗

2s complement width 6.

$$\begin{array}{r} -32\ 16\ 8\ 4\ 2\ 1 \\ [1\ 1\ 0\ 1\ 0\ 0]_{2c,6} \quad \text{summand1} \\ +[0\ 0\ 0\ 1\ 0\ 1]_{2c,6} \quad \text{summand2} \\ \hline 1\ 1\ 1\ 0\ 0\ 1 \quad \text{result} \\ -32\ 16\ 8\ \quad\ 1 \end{array}$$

−12 + 5 = −7

expected result? ☺

Can fixed-width addition be used to calculate $17 + (-17) = 0$? Hint: use 6 bits and 2s complement.

Not with base 2 expansion b/c can't represent negative integers.

Not with sign magnitude b/c column-by-column calculation won't give expected result, eg. with width 6

$$\begin{array}{r} -32\ 16\ 8\ 4\ 2\ 1 \\ [0\ 1\ 0\ 0\ 0\ 1]_{2c,6} \\ [1\ 0\ 1\ 1\ 1\ 1]_{2c,6} \\ \hline [0\ 0\ 0\ 0\ 0\ 0]_{2c,6} \end{array}$$

$$\begin{array}{r} 16\ 8\ 4\ 2\ 1 \\ 0\ 1\ 0\ 0\ 0\ 1 \\ +\ 1\ 1\ 0\ 0\ 0\ 1 \\ \hline 0\ 0\ 0\ 0\ 1\ 0 \end{array}$$ NOT ZERO

Yes, 2s complement width 6

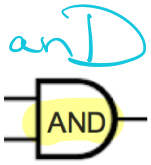
CC BY-NC-SA 2.0

We'll do this on Monday...

In a **combinatorial circuit** (also known as a **logic circuit**), we have **logic gates** connected by **wires**. The inputs to the circuits are the values set on the input wires: possible values are 0 (low) or 1 (high). The values flow along the wires from left to right. A wire may be split into two or more wires, indicated with a filled-in circle (representing solder). Values stay the same along a wire. When one or more wires flow into a gate, the output value of that gate is computed from the input values based on the gate's definition table. Outputs of gates may become inputs to other gates.

Inputs		Output
x	y	x AND y
1	1	1
1	0	0
0	1	0
0	0	0

2^2



of rows = # of possible settings of values to the inputs

2^2

X : exclusive or

Inputs		Output
x	y	x XOR y
1	1	0
1	0	1
0	1	1
0	0	0



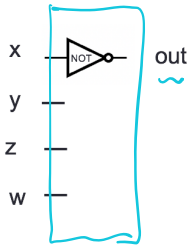
Input	Output
x	NOT x
1	0
0	1

$2 = 2$ #columns



of columns = # of inputs

Example digital circuit:



4 inputs

1 output

Of the 16 possible settings consider:

Output when $x = 1, y = 0, z = 0, w = 1$ is 0

Output when $x = 1, y = 1, z = 1, w = 1$ is 0

Output when $x = 0, y = 0, z = 0, w = 1$ is 1

Draw a logic circuit with inputs x and y whose output is always 0. *Can you use exactly 1 gate?*

CC BY-NC-SA 2.0